



European IT Certification Curriculum Self-Learning Preparatory Materials

EITC/AI/MLP
Machine Learning with Python



This document constitutes European IT Certification curriculum self-learning preparatory material for the EITC/AI/MLP Machine Learning with Python programme.

This self-learning preparatory material covers requirements of the corresponding EITC certification programme examination. It is intended to facilitate certification programme's participant learning and preparation towards the EITC/AI/MLP Machine Learning with Python programme examination. The knowledge contained within the material is sufficient to pass the corresponding EITC certification examination in regard to relevant curriculum parts. The document specifies the knowledge and skills that participants of the EITC/AI/MLP Machine Learning with Python certification programme should have in order to attain the corresponding EITC certificate.

Disclaimer

This document has been automatically generated and published based on the most recent updates of the EITC/AI/MLP Machine Learning with Python certification programme curriculum as published on its relevant webpage, accessible at:

<https://eitca.org/certification/eitc-ai-mlp-machine-learning-with-python/>

As such, despite every effort to make it complete and corresponding with the current EITC curriculum it may contain inaccuracies and incomplete sections, subject to ongoing updates and corrections directly on the EITC webpage. No warranty is given by EITCI as a publisher in regard to completeness of the information contained within the document and neither shall EITCI be responsible or liable for any errors, omissions, inaccuracies, losses or damages whatsoever arising by virtue of such information or any instructions or advice contained within this publication. Changes in the document may be made by EITCI at its own discretion and at any time without notice, to maintain relevance of the self-learning material with the most current EITC curriculum. The self-learning preparatory material is provided by EITCI free of charge and does not constitute the paid certification service, the costs of which cover examination, certification and verification procedures, as well as related infrastructures.

EITC/AI/MLP MACHINE LEARNING WITH PYTHON DIDACTIC MATERIALS**LESSON: REGRESSION****TOPIC: REGRESSION TRAINING AND TESTING****INTRODUCTION**

Artificial Intelligence - Machine Learning with Python - Regression - Regression training and testing

Regression is a fundamental concept in machine learning that involves predicting continuous values based on input data. In this didactic material, we will explore the process of regression training and testing using Python. We will discuss the steps involved in training a regression model, evaluating its performance, and making predictions on new data.

1. Data Preparation:

Before training a regression model, it is important to prepare the data appropriately. This involves cleaning the data, handling missing values, and transforming categorical variables if necessary. Additionally, the data should be split into training and testing sets to assess the model's performance accurately.

2. Choosing a Regression Algorithm:

Python offers several regression algorithms, each with its own strengths and weaknesses. The choice of algorithm depends on the nature of the problem and the characteristics of the data. Some commonly used regression algorithms in Python include linear regression, polynomial regression, support vector regression, and decision tree regression.

3. Training the Regression Model:

To train a regression model in Python, we need to fit the algorithm to the training data. This involves finding the optimal values for the model's parameters based on the input features and target variable. The process of training aims to minimize the difference between the predicted values and the actual values in the training set.

4. Evaluating Model Performance:

After training the regression model, it is important to evaluate its performance to determine its accuracy and reliability. Various metrics can be used for this purpose, such as mean squared error (MSE), root mean squared error (RMSE), mean absolute error (MAE), and R-squared. These metrics provide insights into how well the model fits the training data and its ability to generalize to new data.

5. Testing the Regression Model:

Once the model is trained and evaluated, it can be tested on the testing set to assess its performance on unseen data. The testing process involves inputting the testing features into the trained model and comparing the predicted values with the actual values. This allows us to measure the model's predictive accuracy and determine if it is overfitting or underfitting the data.

6. Making Predictions:

After the regression model is trained and tested, it can be used to make predictions on new, unseen data. By inputting the relevant features into the model, it will produce predicted values based on the learned patterns from the training data. These predictions can be used for various applications, such as forecasting future trends, estimating values, or making informed decisions.

Regression training and testing are essential steps in developing accurate and reliable machine learning models. By carefully preparing the data, choosing the appropriate regression algorithm, training the model, evaluating its performance, and testing it on unseen data, we can build robust regression models using Python. These models can then be used to make predictions and gain valuable insights from the data.

DETAILED DIDACTIC MATERIAL

In this tutorial, we will focus on regression training and testing in the context of machine learning with Python. Regression is a supervised learning technique used to predict continuous numerical values based on input features. We will be using the scikit-learn library, which provides a wide range of machine learning algorithms and tools.

Before we begin, let's make some necessary imports. We will import numpy as NP and scikit-learn's preprocessing module. Numpy is a powerful computing library that allows us to work with arrays, while preprocessing provides various data scaling and transformation methods. Additionally, we will import cross-validation and support vector machine (SVM) modules, which we will use later in the tutorial.

To start, we need to define our features and labels. Features, denoted as x , represent the input variables, while labels, denoted as y , represent the output variable we want to predict. We will use a numpy array to store the features, obtained by dropping the label column from our dataset. Similarly, we will store the labels in a numpy array.

Next, we will scale our features using the `preprocessing.scale` function. Scaling the data is typically done to ensure that the features are within a specific range, often between -1 and 1. This can improve accuracy and processing speed. It's important to note that if we plan to use the classifier in real-time on new data, we need to scale the new values alongside the training data.

After scaling, we will redefine our features array, x , to include only the points for which we have corresponding labels, y . This ensures that our dataset is aligned properly. We will also drop any rows with missing values using the `df.dropna` method.

Finally, we will print the lengths of x and y to verify that we have the correct number of data points.

We have covered the initial steps of regression training and testing. We imported the necessary libraries, defined our features and labels, scaled the features, and ensured the alignment of our dataset. In the next tutorial, we will continue with the regression process.

In the process of regression training and testing in machine learning with Python, it is important to create training and testing sets. To do this, we can use the `train_test_split` function from the scikit-learn library. The function takes in the features (x) and labels (y), as well as the desired test size (in this case, 20% or 0.2). The function shuffles the data while keeping the features and labels connected, and outputs the training and testing data for both x and y .

Once we have the training data, we can proceed to fitting a classifier. In this example, we will use linear regression as the classifier. To fit the classifier, we use the `fit()` function and pass in the features (x_{train}) and labels (y_{train}). This trains the classifier on the training data.

After fitting the classifier, it is important to test its performance. This is done by using the `score()` function, which is synonymous with testing. We pass in the testing features (x_{test}) and labels (y_{test}) to get the accuracy score. It is worth noting that training and testing on separate data is important to avoid overfitting, where the classifier simply memorizes the training data and performs poorly on new data.

In this example, the accuracy score obtained is 0.96, indicating a 96% accuracy in predicting the price with a one-day shift. The accuracy score is calculated using squared error, which will be explained in detail in the upcoming explanation of linear regression.

It is also possible to use other algorithms besides linear regression. For example, support vector regression (SVR) can be used. Switching to SVR is as simple as changing the classifier to SVR in the code. However, it is important to note that the performance of different algorithms can vary significantly. In this example, SVR performs much worse than linear regression, with an accuracy score of only 0.51. It is worth experimenting with different algorithms and their parameters, such as different kernels, to improve performance.

Regression training and testing in machine learning with Python involves creating training and testing sets, fitting a classifier, and evaluating its performance using accuracy scores. It is important to choose the right algorithm and parameters to achieve accurate predictions.

Support Vector Regression is not the focus of this tutorial series. However, it is important to understand the concept of a kernel, which will be explained in the support vector machines tutorial. This example demonstrates how easy it is to switch between different algorithms for regression, classification, or clustering. It is recommended to test different algorithms to find the most suitable one for your task.

When working with various algorithms, it is essential to refer to the documentation. For instance, when using linear regression, you should check if the algorithm supports threading. Threading allows running multiple jobs or threads simultaneously. Linear regression can be threaded easily, unlike support vector machines, which require more complex techniques. By threading linear regression, you can perform parallel operations, resulting in faster training.

To determine if an algorithm can be threaded, you can search for it on Google and look for the "N Jobs" parameter. This parameter indicates the number of jobs or threads that can be run concurrently. By default, linear regression runs with only one job. However, you can specify a higher number to run multiple jobs simultaneously and increase the speed of training. Alternatively, you can use "-1" to utilize as many jobs as possible based on your processor's capabilities.

It is important to consider your computer's processing power while following this tutorial series. If you have an older computer or a laptop, some operations may take longer to run. In such cases, it might be beneficial to use a server or more powerful hardware, especially when working with deep learning algorithms.

Understanding the threading capabilities of different algorithms is important. Linear regression, as well as many other algorithms, can be highly threaded, allowing for efficient parallel operations. As processing power increases, the ability to scale calculations and methodologies becomes increasingly valuable.

In the next tutorial, we will discuss predicting future values using Scikit-learn. Following that, we will consider linear regression, providing a detailed breakdown and implementation. If you have any questions or concerns, feel free to leave them in the comments section. Thank you for your support and stay tuned for the upcoming tutorials.

EITC/AI/MLP MACHINE LEARNING WITH PYTHON - REGRESSION - REGRESSION TRAINING AND TESTING - REVIEW QUESTIONS:**HOW DO WE CREATE TRAINING AND TESTING SETS IN REGRESSION TRAINING AND TESTING?**

To create training and testing sets in regression training and testing, we follow a systematic process that involves splitting the available data into two separate datasets: the training set and the testing set. This division allows us to train our regression model on a subset of the data and evaluate its performance on unseen data.

The first step in creating training and testing sets is to determine the desired ratio between the two. This ratio depends on various factors, such as the size of the dataset, the complexity of the problem, and the available computational resources. A common practice is to allocate around 70-80% of the data for training and the remaining 20-30% for testing. However, this ratio can be adjusted based on the specific requirements of the problem at hand.

Once the ratio is decided, we randomly split the data into the training and testing sets. Randomness is important to ensure that the data is representative of the overall distribution and to avoid any bias in the model's performance evaluation. In Python, we can achieve this using various libraries, such as scikit-learn or numpy.

Let's consider an example to illustrate the process. Suppose we have a dataset containing information about house prices, including features such as the number of bedrooms, the area, and the location. Our goal is to build a regression model that can predict the price of a house based on these features.

We start by importing the necessary libraries and loading the dataset into memory:

1.	<code>import pandas as pd</code>
2.	<code>from sklearn.model_selection import train_test_split</code>
3.	<code># Load the dataset</code>
4.	<code>data = pd.read_csv('house_prices.csv')</code>

Next, we separate the input features (X) from the target variable (y):

1.	<code>X = data.drop('price', axis=1)</code>
2.	<code>y = data['price']</code>

Now, we can split the data into training and testing sets using the `train_test_split()` function from scikit-learn:

1.	<code>X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)</code>
----	--

In this example, we have allocated 20% of the data for testing (`test_size=0.2`). The `random_state` parameter ensures reproducibility by fixing the random seed used for the split.

After the split, we have four separate datasets: `X_train` (input features for training), `X_test` (input features for testing), `y_train` (target variable for training), and `y_test` (target variable for testing). These datasets are ready to be used for training and evaluating our regression model.

It is important to note that the training and testing sets should be representative of the overall data distribution and should not contain any overlapping samples. This ensures that our model can generalize well to unseen data and provides a reliable estimate of its performance.

Creating training and testing sets in regression training and testing involves randomly splitting the available data into two separate datasets: the training set, used to train the regression model, and the testing set, used to evaluate its performance. This process helps us assess how well our model generalizes to unseen data and

provides valuable insights into its effectiveness.

WHAT IS THE PURPOSE OF FITTING A CLASSIFIER IN REGRESSION TRAINING AND TESTING?

Fitting a classifier in regression training and testing serves an important purpose in the field of Artificial Intelligence and Machine Learning. The primary objective of regression is to predict continuous numerical values based on input features. However, there are scenarios where we need to classify the data into discrete categories rather than predicting continuous values. In such cases, fitting a classifier becomes essential.

The purpose of fitting a classifier in regression training and testing is to transform the regression problem into a classification problem. By doing so, we can leverage the power of classification algorithms to solve the regression task. This approach allows us to utilize a wide range of classifiers that are specifically designed for handling classification problems.

One common technique for fitting a classifier in regression is to discretize the continuous output variable into a set of predefined categories. For instance, if we are predicting house prices, we can divide the price range into categories like "low," "medium," and "high." We can then train a classifier to predict these categories based on the input features such as the number of rooms, location, and square footage.

By fitting a classifier, we can take advantage of various classification algorithms such as decision trees, random forests, support vector machines, and neural networks. These algorithms are capable of handling complex relationships between input features and the target variable. They can learn decision boundaries and patterns in the data to make accurate predictions.

Moreover, fitting a classifier in regression training and testing allows us to evaluate the performance of the regression model in a classification context. We can use well-established evaluation metrics such as accuracy, precision, recall, and F1-score to assess how well the regression model performs when treated as a classifier.

Additionally, fitting a classifier in regression training and testing provides a didactic value. It helps us explore different perspectives and approaches to solving regression problems. By considering the problem as a classification task, we can gain insights into the underlying patterns and relationships in the data. This broader perspective enhances our understanding of the data and can lead to innovative solutions and feature engineering techniques.

To illustrate the purpose of fitting a classifier in regression training and testing, let's consider an example. Suppose we have a dataset containing information about the performance of students, including features such as study hours, attendance, and previous grades. The target variable is the final exam score, which is a continuous value. If we want to predict whether a student will pass or fail based on their final exam score, we can fit a classifier by discretizing the scores into two categories: "pass" and "fail." We can then train a classifier using the input features to predict the pass/fail outcome.

Fitting a classifier in regression training and testing allows us to transform a regression problem into a classification problem. It enables us to leverage the power of classification algorithms, evaluate the performance of the regression model in a classification context, and gain a broader understanding of the data. This approach provides a valuable perspective and opens up new possibilities for solving regression problems.

HOW DO WE EVALUATE THE PERFORMANCE OF A CLASSIFIER IN REGRESSION TRAINING AND TESTING?

In the field of Artificial Intelligence, specifically in Machine Learning with Python, the evaluation of a classifier's performance in regression training and testing is important in order to assess its effectiveness and determine its suitability for a given task. Evaluating a classifier involves measuring its ability to accurately predict continuous values, such as estimating the price of a house or the temperature of a room.

One commonly used metric for evaluating the performance of a regression classifier is the mean squared error (MSE). The MSE calculates the average of the squared differences between the predicted values and the true values. This metric provides a measure of how close the predicted values are to the actual values. A lower MSE

indicates a better fit of the classifier to the data.

Another commonly used metric is the root mean squared error (RMSE), which is the square root of the MSE. The RMSE provides a measure of the average absolute difference between the predicted values and the true values. Like the MSE, a lower RMSE indicates a better fit of the classifier to the data.

Additionally, the coefficient of determination (R-squared) is often used to evaluate the performance of a regression classifier. The R-squared metric measures the proportion of the variance in the dependent variable that can be explained by the independent variables. It ranges from 0 to 1, with 1 indicating a perfect fit of the classifier to the data.

In Python, the scikit-learn library provides a convenient way to evaluate the performance of a regression classifier. The `mean_squared_error` function from the `sklearn.metrics` module can be used to calculate the MSE, while the `r2_score` function can be used to calculate the R-squared value. These functions take the true values and the predicted values as input and return the corresponding metric.

Here is an example of how to evaluate the performance of a regression classifier using the MSE and R-squared metrics in Python:

1.	<code>from sklearn.metrics import mean_squared_error, r2_score</code>
2.	<code># Assuming you have the true values stored in a variable called true_values</code>
3.	<code># and the predicted values stored in a variable called predicted_values</code>
4.	<code>mse = mean_squared_error(true_values, predicted_values)</code>
5.	<code>rmse = np.sqrt(mse)</code>
6.	<code>r2 = r2_score(true_values, predicted_values)</code>
7.	<code>print("Mean Squared Error: ", mse)</code>
8.	<code>print("Root Mean Squared Error: ", rmse)</code>
9.	<code>print("R-squared: ", r2)</code>

Evaluating the performance of a classifier in regression training and testing involves using metrics such as mean squared error, root mean squared error, and coefficient of determination. These metrics provide insights into the accuracy and goodness of fit of the classifier to the data. Python libraries like scikit-learn provide convenient functions to calculate these metrics.

WHY IS IT IMPORTANT TO CHOOSE THE RIGHT ALGORITHM AND PARAMETERS IN REGRESSION TRAINING AND TESTING?

Choosing the right algorithm and parameters in regression training and testing is of utmost importance in the field of Artificial Intelligence and Machine Learning. Regression is a supervised learning technique used to model the relationship between a dependent variable and one or more independent variables. It is widely used for prediction and forecasting tasks. The selection of the appropriate algorithm and its parameters can significantly impact the accuracy and performance of the regression model. In this answer, we will explore the reasons why it is important to make informed choices in algorithm and parameter selection.

Firstly, selecting the right algorithm is essential because different regression algorithms have varying characteristics and assumptions. Each algorithm makes specific assumptions about the underlying data distribution and the relationship between the dependent and independent variables. For example, linear regression assumes a linear relationship between the variables, while decision trees can handle non-linear relationships. By understanding the nature of the problem and the data, one can choose an algorithm that best suits the problem at hand. Using an algorithm that aligns with the data characteristics can lead to better model performance and more accurate predictions.

Secondly, the choice of algorithm can also impact the interpretability of the regression model. Some algorithms, such as linear regression or decision trees, provide easily interpretable coefficients or rules that can help understand the relationship between the variables. On the other hand, complex algorithms like neural networks may provide accurate predictions but lack interpretability. Depending on the requirements of the problem and the stakeholders involved, interpretability may be a critical factor in algorithm selection.

Furthermore, the selection of appropriate parameters for the chosen algorithm is important for achieving optimal model performance. Parameters are values that control the behavior of the algorithm during the training process. Different algorithms have different parameters that need to be set, such as learning rate, regularization strength, or the number of hidden layers in a neural network. Setting these parameters correctly can greatly impact the convergence speed, model complexity, and generalization ability of the regression model.

To choose the right parameters, one can employ techniques such as grid search or random search. Grid search involves exhaustively searching through a predefined set of parameter combinations and selecting the one that yields the best performance. Random search, on the other hand, randomly samples parameter combinations and evaluates their performance. These techniques help in finding the optimal combination of parameters that maximizes the model's accuracy or minimizes the error.

Choosing the right algorithm and parameters is not a one-size-fits-all approach. It requires a deep understanding of the problem, the data, and the characteristics of different algorithms. It is often an iterative process that involves experimentation, evaluation, and fine-tuning. It is important to evaluate the performance of the model using appropriate evaluation metrics such as mean squared error (MSE) or R-squared. By comparing the performance of different algorithms and parameter settings, one can make an informed decision on the best approach.

The choice of algorithm and parameters in regression training and testing is critical for achieving accurate predictions, model interpretability, and optimal performance. Selecting the right algorithm that aligns with the problem and data characteristics, and fine-tuning the parameters, can greatly influence the model's accuracy and generalization ability. It is a process that requires careful consideration, experimentation, and evaluation. By making informed choices, one can build robust regression models that effectively capture the underlying relationships in the data.